

宁波大学
NINGBO UNIVERSITY

信息安全技术实验报告

[LSB 隐写实验]

姓名：	顾凯琪
学号：	226001624
课程名称：	信息安全技术
所在学院：	信息科学与工程学院
专业班级：	22 级计算机 2 班
任课教师：	徐博文 严迪群

实验项目名称	LSB 隐写实验

一、 实验目的与要求：

实验目的：

1. 使用 LSB 隐写方法在文件中进行密信(密信为自己的学号姓名)隐藏；
2. 通过 LSB 方法提取隐藏在文件中的密信

实验要求：

1. 使用 CrypTool 2.1 来实现图片的隐写，并使用 CrypTool 2.1 来提取隐藏在文件中的密信；
2. 使用 python 设计算法来实现图片的隐写和提取；
3. 在网络上下载工具来实现图片的隐写和信息提取。

二、 实验环境：

1. Windows10 系统
2. CrypTool 2.1 软件
3. Python 3.9

三、实验方法（原理、流程图）

3.1 什么是隐写？

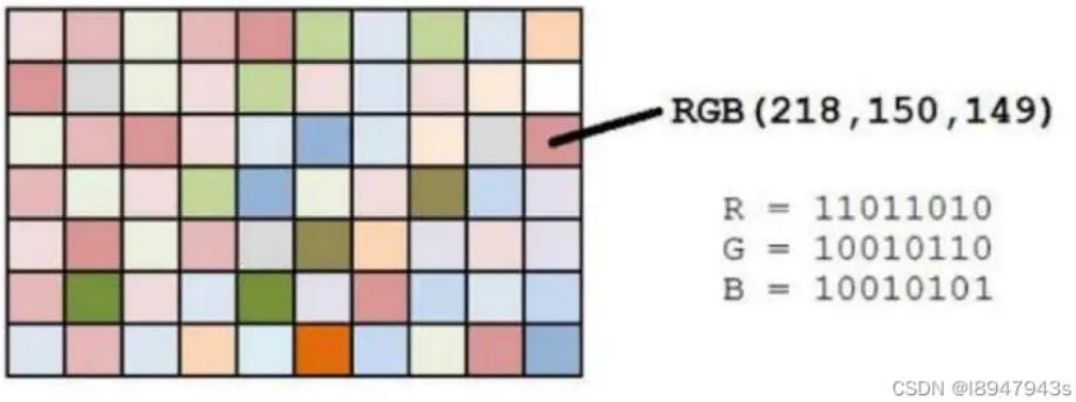
隐写术是一门关于信息隐藏的技巧与科学，所谓信息隐藏指的是不让除预期的接收者之外的任何人知晓信息的传递事件或者信息的内容。隐写术的英文叫做 Steganography，来源于特里特米乌斯的一本讲述密码学与隐写术的著作 Steganographia，该书书名源于希腊语，意为“隐秘书写”。

3.2 什么是图片隐写？

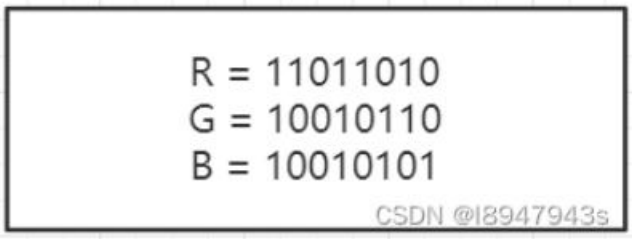
图片隐写术这项技术可以将秘密信息嵌入到图片媒介中而不损坏它的载体的质量。第三方既觉察不到秘密信息的存在，也不知道存在秘密信息。因此密钥、数字签名和私密信息都可以在开放的环境(如 Internet 或者内联网)中安全的传送。简单概括就是信息明明就在眼前，但是你却视而不见。

3.3 LSB 隐写的原理

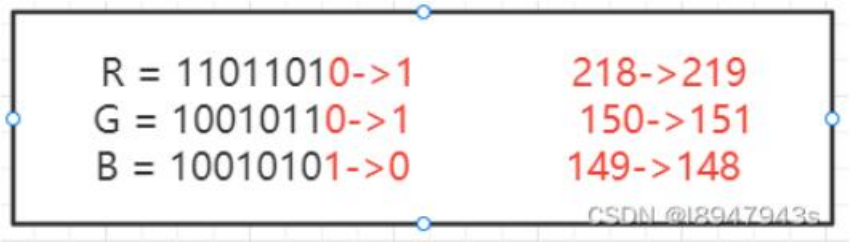
在 png 图片的存储中，每个颜色表示需要有 8bit，即有 256 种颜色，一共包含 256 的三次方个颜色，即 16777216 中颜色，人类的眼睛可以区分约 1,000 万种不同的颜色，剩下无法区分的颜色就有 6777216。



那么什么是 LSB 呢，每个像素点的颜色可以用十进制 0~255 的数值表示，那么上图中的 RGB（218,150,149）表示成二进制为：



LSB 隐写就是修改 RGB 颜色分量的最低二进制位也就是最低有效位（LSB），而人类的眼睛不会注意到这前后的变化，我们仅对 RGB 中的最低位进行修改，如当前像素位点最后一位修改成 1,1，0.则上述 RGB 变成 218->219：

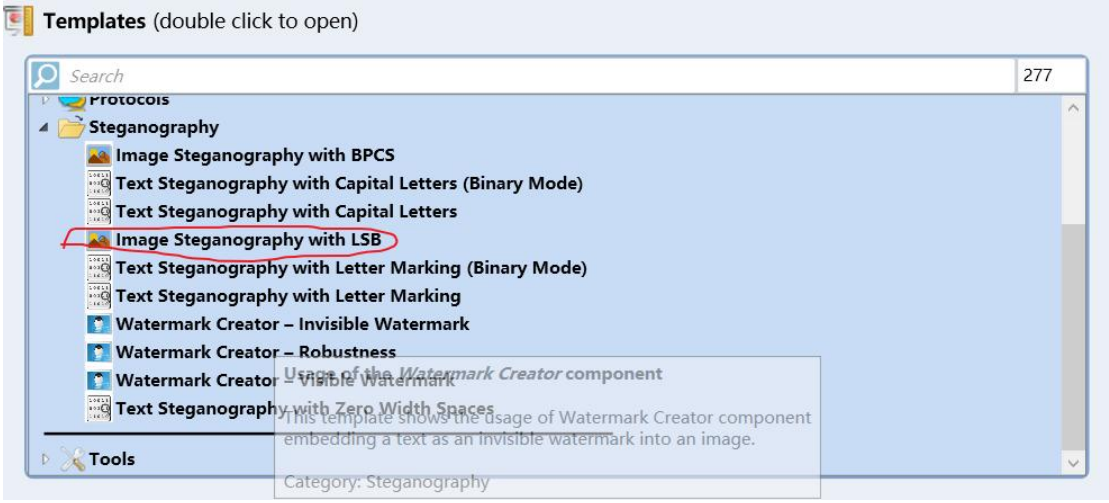


上述变化很难用肉眼察觉到，而且每一个像素位点携带了一位信息，那么我们可以利用八个字节的最低位存储一个比特信息，而该比特信息则可以转化为 ASCII 字符，从而达到隐写信息的目的。

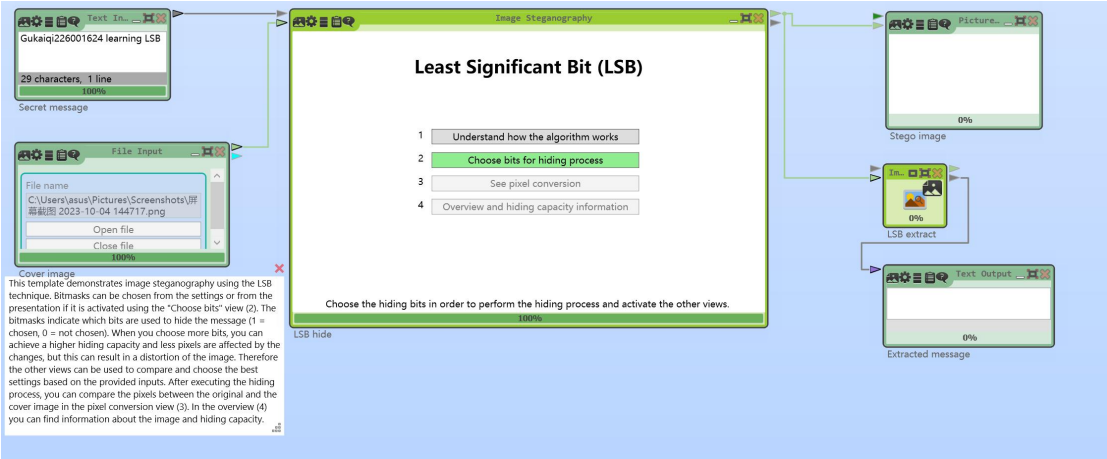
四、实验过程、步骤及内容

4.1 使用 CrypTool 2.1 来模拟实现 LSB 隐写实验

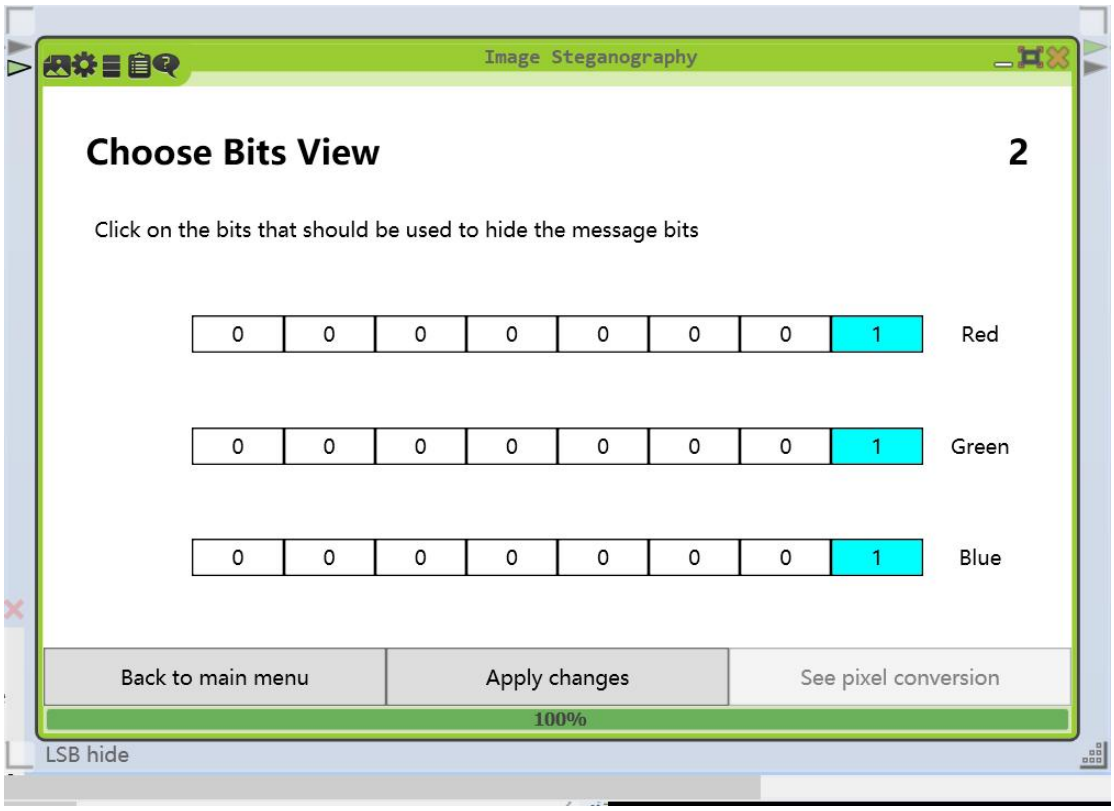
点击 Templates 中的 LSB，打开实例



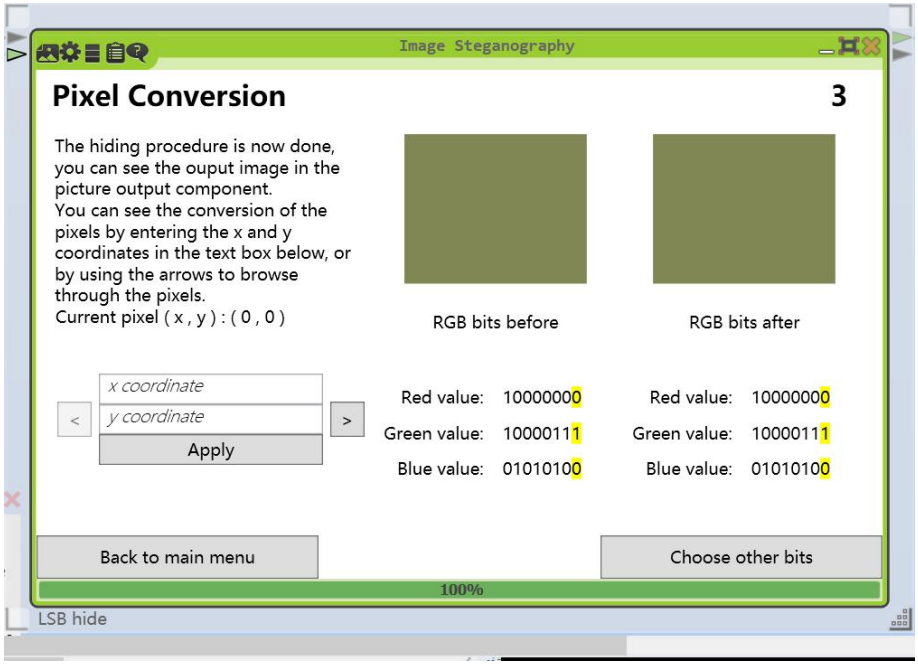
从图中我们可以大致把这个系统分为三个部分，一部分为输入，输入我们需要隐写的信息与存储信息的图片；中间部分为图像处理部分，将隐写的信息进行处理存入图像当中；最后一部分输出我们处理完的图像，并从我们处理完后的图像提取隐写信息。



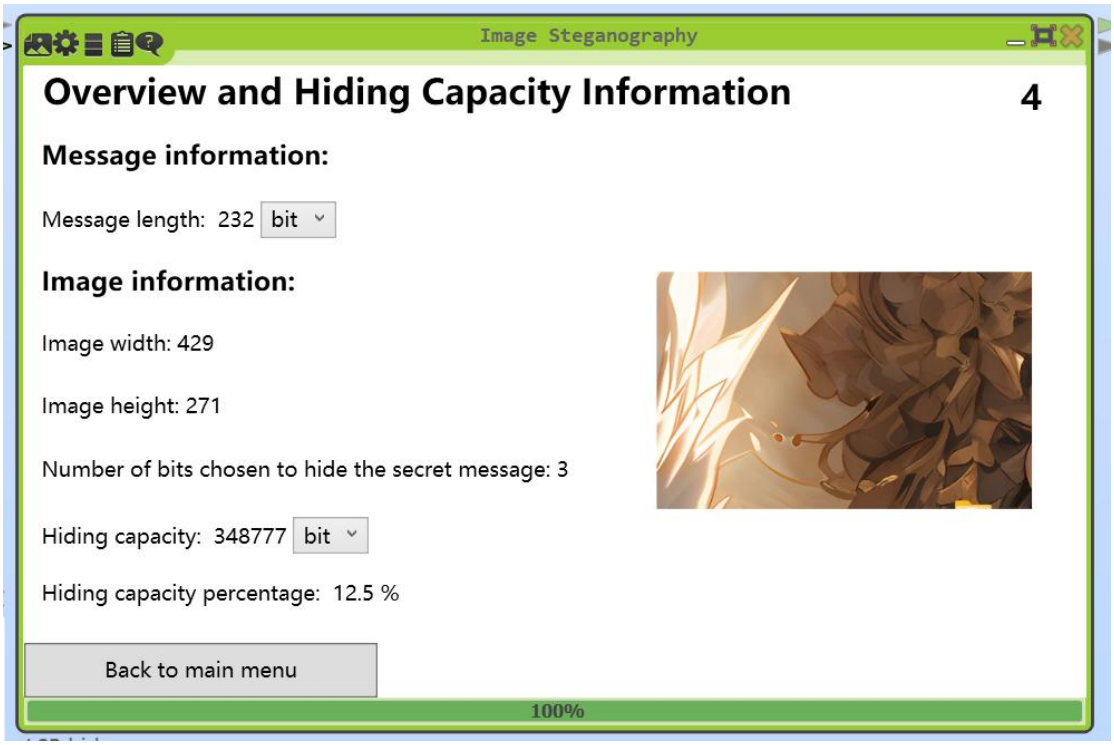
该模板使用 LSB 技术演示了图像隐写术。总共分为四步。第一步运行，理解 LSB 图像隐写技术的用法。第二步选择位用于隐藏信息（1=选择，0=不选择）。如果选择的位数越多，隐藏能力就越强，受变化的像素就越少，但这会导致图像失真。



在执行隐藏过程以后，我们可以在像素转换试图中比较原始图像和隐写处理后的图像像素点的区别。可以输入 x, y 轴的坐标来定位我们想要查看的像素点。



在概览（4）中，我们可以找到有关图像和隐藏容量的信息。



4.2 使用 python 来设计算法脚本实现 LSB 图像隐写功能

4.2.1 使用隐写算法对图片进行加工处理

一、读取图片和文本文件

```python
import numpy as np
import PIL.Image as Image

# 读取图片的像素信息
picture = Image.open('./pic/pic.jpg')
pic_data = np.array(picture)

# 读取要隐写的文件，长度为4893
with open('./pic/secret.txt', encoding="utf-8") as file:
    secrets = file.read()

# 将图片拷贝一份，作为最终的图片数据
im_data = np.array(picture.copy()).ravel().tolist()
```

二、定义 LSB 隐写函数 cover_lsb

```python
def cover_lsb(bin_index, data):
    '''
    :param bin_index:  当前字符的ascii的二进制
    :param data: 取出数组像素的八个数值
    :return: LSB隐写后的字符
    '''
    res = []
    for i in range(8):
        data_i_bin = bin(data[i])[2:].zfill(8)
        if bin_index[i] == '0':
            data_i_bin = data_i_bin[0:7] + '0'
        elif bin_index[i] == '1':
            data_i_bin = data_i_bin[0:7] + '1'
        res.append(int(data_i_bin, 2))
    return res
```

三、隐写文本文件到图片

```python
pic_idx = 0
# 采用LSB隐写技术，横向取数据，每次取9个数据，改变8个像素最低位
res_data = []
for i in range(len(secrets)):
    # 拿到隐写文件的字符ascii数值，并转换为二进制,填充成八位
    index = ord(secrets[i])
    bin_index = bin(index)[2:].zfill(8)
    # 对数据进行LSB隐写，替换操作
    res = cover_lsb(bin_index, im_data[pic_idx * 8: (pic_idx + 1) * 8])
    pic_idx += 1
    res_data += res
```

四、处理修改后的图片

```python
# 对剩余未填充的数据进行补充填充，防止图像无法恢复
res_data += im_data[pic_idx * 8:]
```

五、保存修改后的图片

```python
# 将新生成的文件进行格式转换并保存，此处一定保存为压缩的png文件
new_im_data = np.array(res_data).astype(np.uint8).reshape((pic_data.shape))
res_im = Image.fromarray(new_im_data)
res_im.save('./pic/res_encode.png')
```

4.2.2 对隐写完的图片进行信息提取

一、读取图片和文本文件

```
import numpy as np
import PIL.Image as Image

# 打开隐写文件
picture = Image.open('./pic/res_encode.png')
pic_datas = np.array(picture).ravel().tolist()

# 字符的长度为4893
with open('./pic/secret.txt', encoding="utf-8") as file:
    secrets = file.read()
```

二、打印字符长度并复制图片数据

```
str_len = len(secrets)
print('字符的长度为：', str_len)

# 将图片拷贝一份，作为最终的图片数据
im_data = np.array(picture.copy()).ravel().tolist()
```

三、定义 LSB 提取函数 lsb_decode

```
def lsb_decode(data):
    '''
    :param bin_index:  当前字符的ascii的二进制
    :param data: 取出数组像素的八个数值
    :return: LSB隐写后的字符
    '''
    str = ''
    for i in range(len(data)):
        print(bin(data[i])[2:])
        data_i_bin = bin(data[i])[2:][-1]
        str += data_i_bin
    return str
```

四、转换为 ASCII 码

```
pic_idx = 0
# 采用LSB隐写技术，横向取数据，每次取9个数据，改变8个像素最低位
res_data = []

for i in range(len(secrets)):
    # 拿到第i个数据,转换成二进制
    data = im_data[i * 8: (i + 1) * 8]
    data_int = lsb_decode(data)
    # 找到最低位
    res_data.append(int(data_int, 2))

# 将二进制数据转换成ASCII
str_data = ''
for i in res_data:
    temp = chr(i)
    str_data += temp
print(str_data)
```

4.3 网络上下载工具(zsteg)来实现图片的隐写和信息提取

steg 可以检测 PNG 和 BMP 图片里的隐写数据。

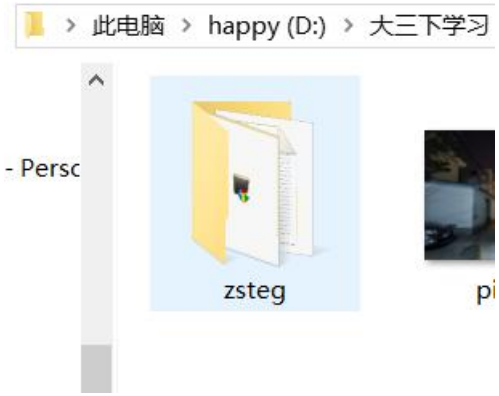
目前，zsteg 支持检测：

4.3.1 安装

首先将文件下载到本地:

这个会下载到当前打开的文件夹下

git clone http://www.github.com/zed-0xff/zsteg



要换国内源

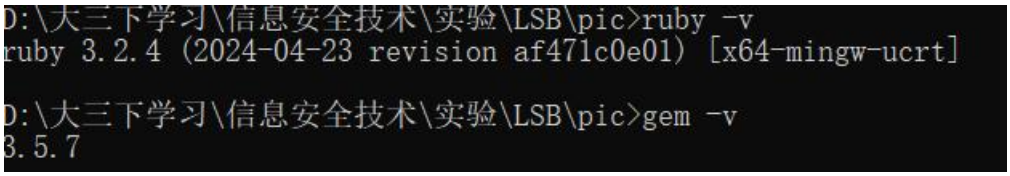
1.查看现有源：

```
gem sources -l
```

此时我们会发现我们的电脑中还未安装 gem,这是因为我们需要先安装 Ruby 和 RubyGems,在此笔者直接进入官网（下载 (rubyinstaller.org)）进行安装。



输入 ruby -v 和 gem -v 可以检查我们是否已经安装完成



要换国内源

1.查看现有源

```
gem sources -l
```

2.删除现有源：

```
gem sources --remove https://rubygems.org/
```

3.添加新源：

```
gem sources -a http://gems.ruby-china.com/
```

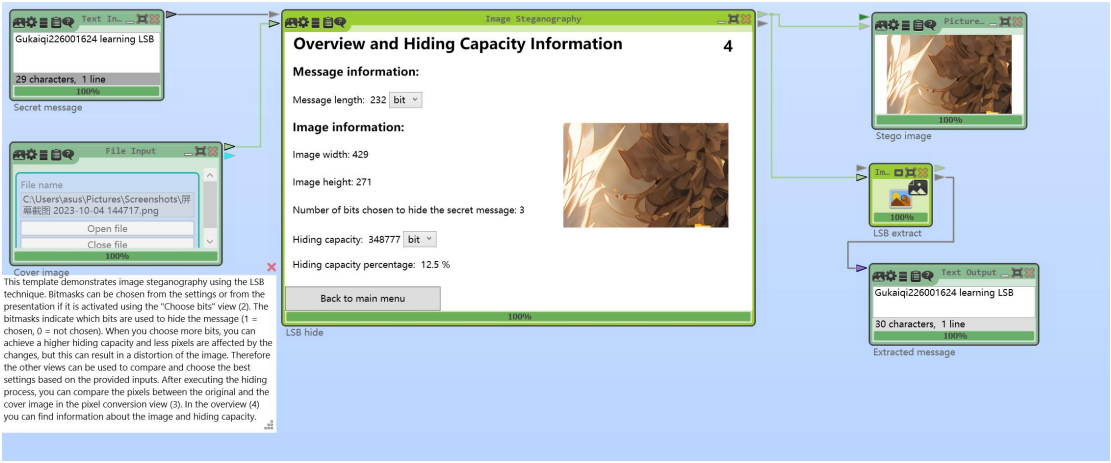
安装 zsteg：

```
gem install zsteg
```

五、实验数据（现象）处理分析

5.1 使用 CrypTool 2.1 来模拟实现 LSB 隐写实验

我们在 Text Input 文本框中输入学号和姓名来代表我们想要隐写的数据，File Input 中加载我们想要进行隐写的图片，运行后我们可以在 picture output 中看到经过隐写处理后的图片。输出的图片经过 LSB extract 处理后可以提取出我们所隐写的数据内容。

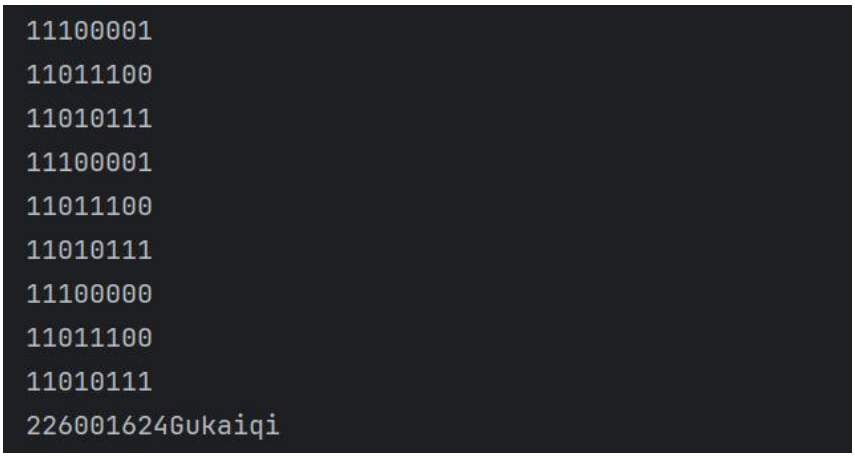


5.2 使用 python 来设计算法脚本实现 LSB 图像隐写功能测试结果

我们使用 LSB_ENCODE 脚本成功实现了对图片的隐写处理



接着我们使用 LSB_DECODE 脚本成功提取了隐写处理中的内容



5.3 网络上下载工具(zsteg)来实现图片的隐写和信息提取测试

通过 zsteg -h 命令我们可以看到我们已经成功地安装了 zsteg 工具

```
D:\大三下学习\信息安全技术\实验\LSB\pic\zsteg>zsteg -h
Usage: zsteg [options] filename.png [param_string]

    -a, --all                        try all known methods
    -E, --extract NAME               extract specified payload, NAME is like '1b,rgb,lsb'

Iteration/extraction params:
    -o, --order X                    pixel iteration order (default: 'auto')
                                     valid values: ALL,xy,yx,XY,YX,xY,Xy,bY,...
    -c, --channels X                 channels (R/G/B/A) or any combination, comma separated
                                     valid values: r,g,b,a,rg,bgr,rgba,r3g2b3,...
    -b, --bits N                     number of bits, single int value or '1,3,5' or range '1-8'
                                     advanced: specify individual bits like '00001110' or '0x88'
        --lsb                        least significant bit comes first
        --msb                        most significant bit comes first
    -P, --prime                      analyze/extract only prime bytes/pixels
        --shift N                    prepend N zero bits
        --invert                     invert bits (XOR 0xff)
        --pixel-align                pixel-align hidden data

Analysis params:
    -l, --limit N                    limit bytes checked, 0 = no limit (default: 256)

        --[no-]file                  use 'file' command to detect data type (default: YES)
        --no-strings                 disable ASCII strings finding (default: enabled)
    -s, --strings X                  ASCII strings find mode: first, all, longest, none
                                     (default: first)
    -n, --min-str-len X              minimum string length (default: 8)

    -v, --verbose                    Run verbosely (can be used multiple times)
    -q, --quiet                      Silent any warnings (can be used multiple times)
    -C, --[no-]color                 Force (or disable) color output (default: auto)

PARAMS SHORTCUT
        zsteg fname.png 2b,b,lsb,xy  ==>  --bits 2 --channel b --lsb --order xy

D:\大三下学习\信息安全技术\实验\LSB\pic\zsteg>zsteg pcat.png
[!] #<Errno::ENOENT: No such file or directory @ rb_sysopen - pcat.png>
```

但是接下来我们使用该工具对我们隐写处理后的图片进行检测时却产生了报错

```
D:\大三下学习\信息安全技术\实验\LSB\pic>zsteg res_encode.png
C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg/file_cmd.rb:106:in `stop!': undefined method `close' for nil:NilCla
ss (NoMethodError)

      @stdin.close
             ^^^^^^

        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg/checker.rb:138:in `ensure in check'
        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg/checker.rb:138:in `check'
        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg/cli/cli.rb:258:in `check'
        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg/cli/cli.rb:172:in `block (2 levels) in run'
        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg/cli/cli.rb:168:in `each'
        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg/cli/cli.rb:168:in `block in run'
        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg/cli/cli.rb:161:in `each'
        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg/cli/cli.rb:161:in `each_with_index'
        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg/cli/cli.rb:161:in `run'
        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg.rb:26:in `run'
        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/bin/zsteg:8:in `<top (required)>'
        from C:/Ruby32-x64/bin/zsteg:32:in `load'
        from C:/Ruby32-x64/bin/zsteg:32:in `<main>'
C:/Ruby32-x64/lib/ruby/3.2.0/open3.rb:222:in `spawn': No such file or directory - file -n -b -f - (Errno::ENOENT)
        from C:/Ruby32-x64/lib/ruby/3.2.0/open3.rb:222:in `popen_run'
        from C:/Ruby32-x64/lib/ruby/3.2.0/open3.rb:103:in `popen3'
        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg/file_cmd.rb:67:in `start!'
        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg/checker.rb:73:in `check'
        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg/cli/cli.rb:258:in `check'
        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg/cli/cli.rb:172:in `block (2 levels) in run'
        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg/cli/cli.rb:168:in `each'
        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg/cli/cli.rb:168:in `block in run'
        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg/cli/cli.rb:161:in `each'
        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg/cli/cli.rb:161:in `each_with_index'
        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg/cli/cli.rb:161:in `run'
        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg.rb:26:in `run'
        from C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/bin/zsteg:8:in `<top (required)>'
        from C:/Ruby32-x64/bin/zsteg:32:in `load'
        from C:/Ruby32-x64/bin/zsteg:32:in `<main>'

D:\大三下学习\信息安全技术\实验\LSB\pic>gem -v
3.5.7
```

我查阅诸多资料仍然找不到解决方案，更新 gem 依旧不行。可能因为我采用的是 Windows 系统原因，有可能该工具只适配于 Linux 系统。

```
D:\大三下学习\信息安全技术\实验\LSB\pic>gem update --system
Updating rubygems-update
Fetching rubygems-update-3.5.7.gem
Successfully installed rubygems-update-3.5.7
Parsing documentation for rubygems-update-3.5.7
Installing ri documentation for rubygems-update-3.5.7
Done installing documentation for rubygems-update after 1 seconds
Parsing documentation for rubygems-update-3.5.7
Done installing documentation for rubygems-update after 0 seconds
Installing RubyGems 3.5.7
  Successfully built RubyGem
  Name: bundler
  Version: 2.5.7
  File: bundler-2.5.7.gem
Bundler 2.5.7 installed
RubyGems 3.5.7 installed
Regenerating binstubs
Regenerating plugins
Parsing documentation for rubygems-3.5.7
Installing ri documentation for rubygems-3.5.7

= 3.5.7 / 2024-03-22
```

```
D:\大三下学习\信息安全技术\实验\LSB\pic>zsteg res_encode.png
C:/Ruby32-x64/lib/ruby/gems/3.2.0/gems/zsteg-0.2.13/lib/zsteg/file_cmd.rb:106:in `stop!': undefined method `close' for nil:NilCla
ss (NoMethodError)
```

六、实验结论

本次实验以 LSB（最低有效位）隐写技术为核心，采用 CrypTool 2.1 图形化工具、Python 编程以及 zsteg 第三方隐写检测工具，全面探索了数字图像中信息隐藏与提取的实现方式。通过实验，成功实现了将个人学号与姓名嵌入图像中并正确提取，验证了 LSB 隐写在静态图像中的可行性与实用性。

其中，CrypTool 2.1 提供了直观的操作平台，有助于理解隐写原理与信息嵌入过程；Python 脚本的编写提升了算法实现与调试能力，展现了隐写自动化处理的灵活性；zsteg 工具虽然安装成功，但因系统兼容性问题未能顺利完成检测任务，提示我们在工具选择与平台适配方面仍需深入考虑。

总体来看，本实验不仅加深了对信息隐藏技术的理解，也培养了跨工具、跨语言解决问题的能力。未来，可进一步研究如何结合加密与隐写技术提升信息隐藏的安全性，或探索在音频、视频等多媒体载体中的隐写应用。此外，对隐写检测与对抗技术的学习也将是信息安全领域的重要方向。

七、教师批阅意见： 成绩评定： 教师签字： 年　　月　　日
八、备注：